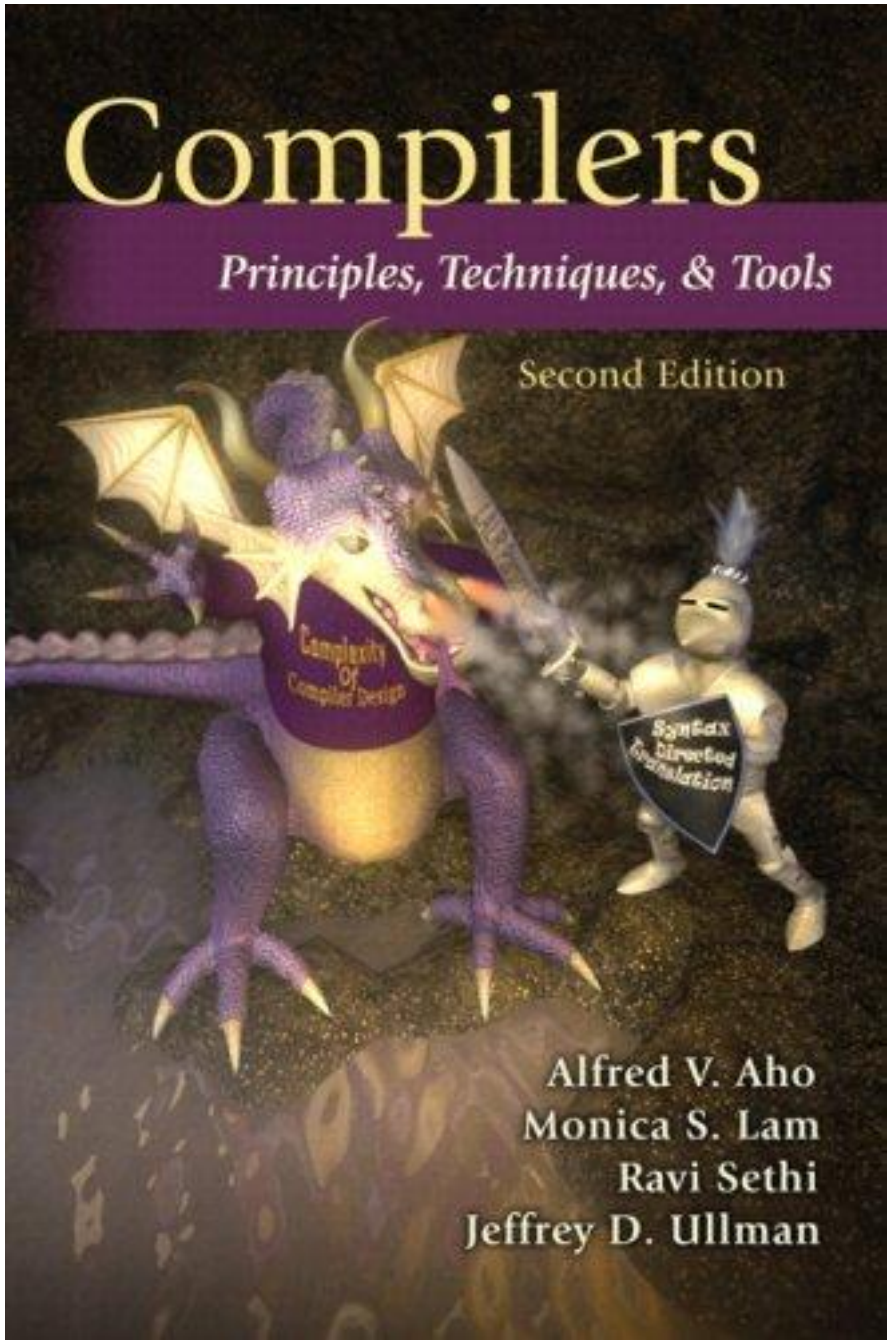
四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn



Chapter 3

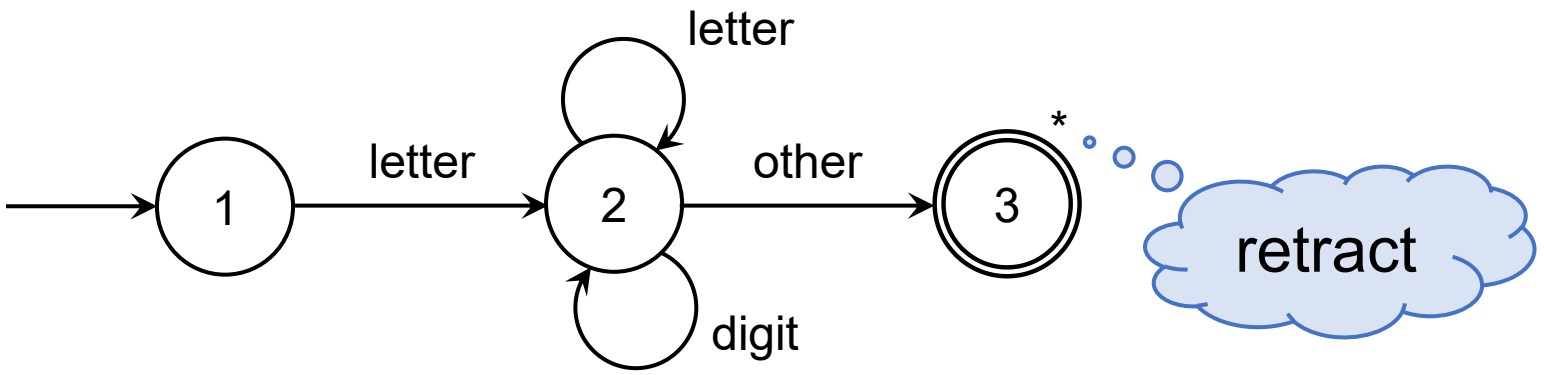
Lexical Analysis

3.4 Recognition of Tokens

3.3 Specification of Tokens vs. 3.4 Recognition of Tokens

- Static structure vs. Dynamic process
- identifiers

$$id = letter\,(letter \mid digit)^*$$



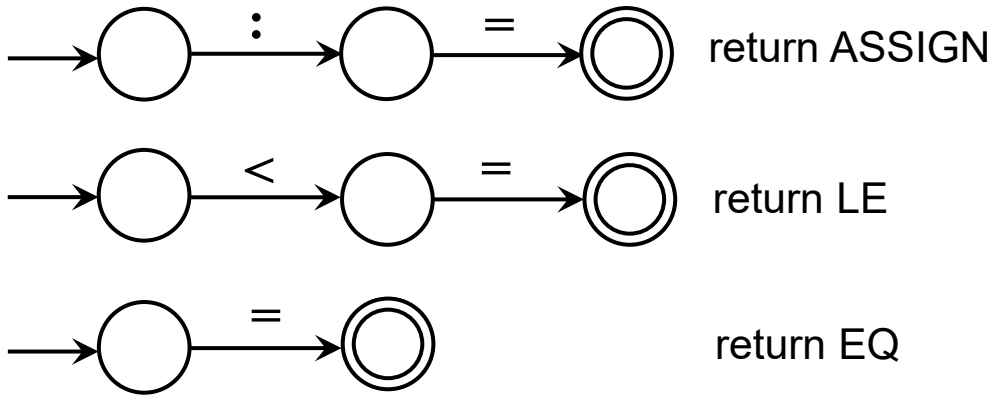
- state
- edge (label)
- start state
- accepting state

3.4.1 Transition Diagrams

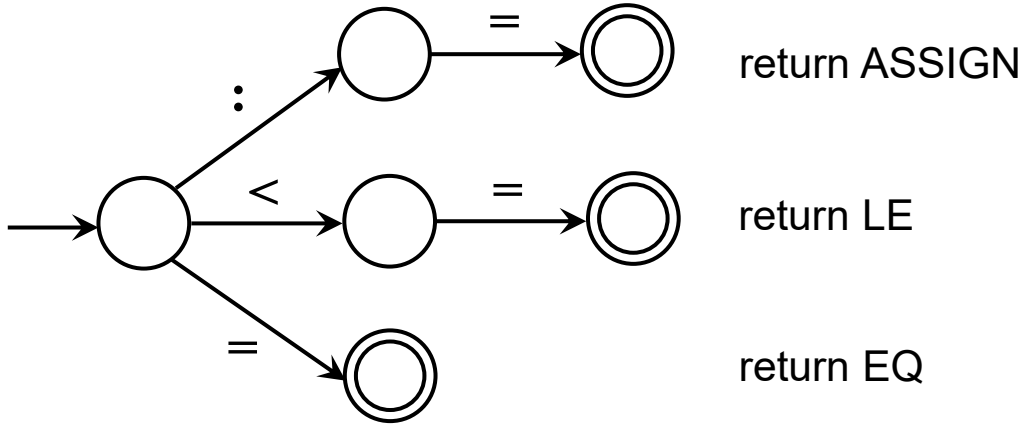
- 状态转移图可用于识别和某个模式匹配的字符串/词素。其中每个状态代表一个可能在这个过程中出现的情况。
- Type of tokens
  - operators & delimiters
  - identifiers
  - keywords
  - constants
  - whitespace: newline, blank, tab, comment

Recognition of Operators

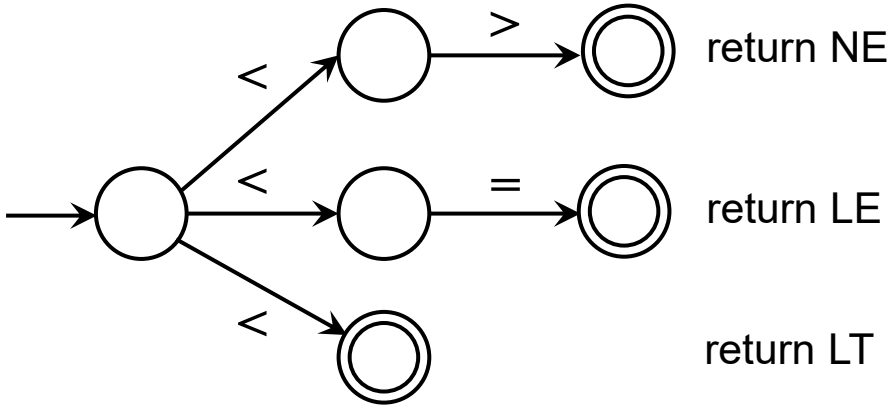
- :=, <=, and =

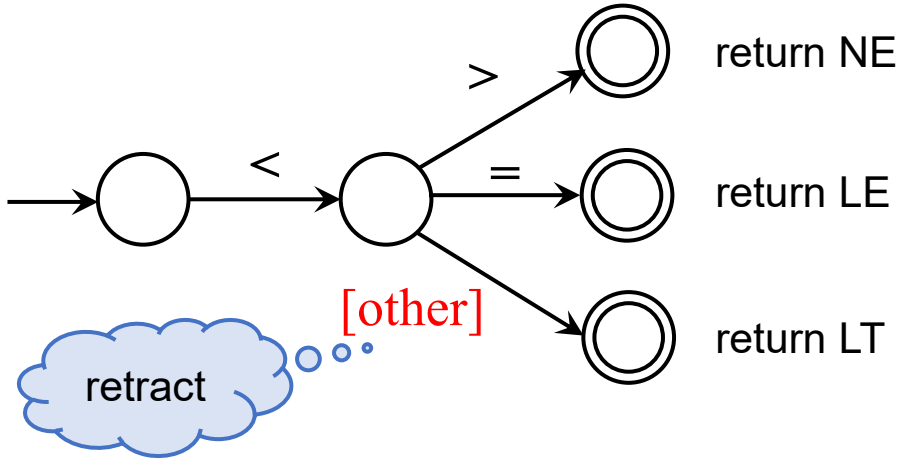


- Combine the tokens into one diagram



- <>, <=, and <





Example 3.9

- Combine == into the diagram?

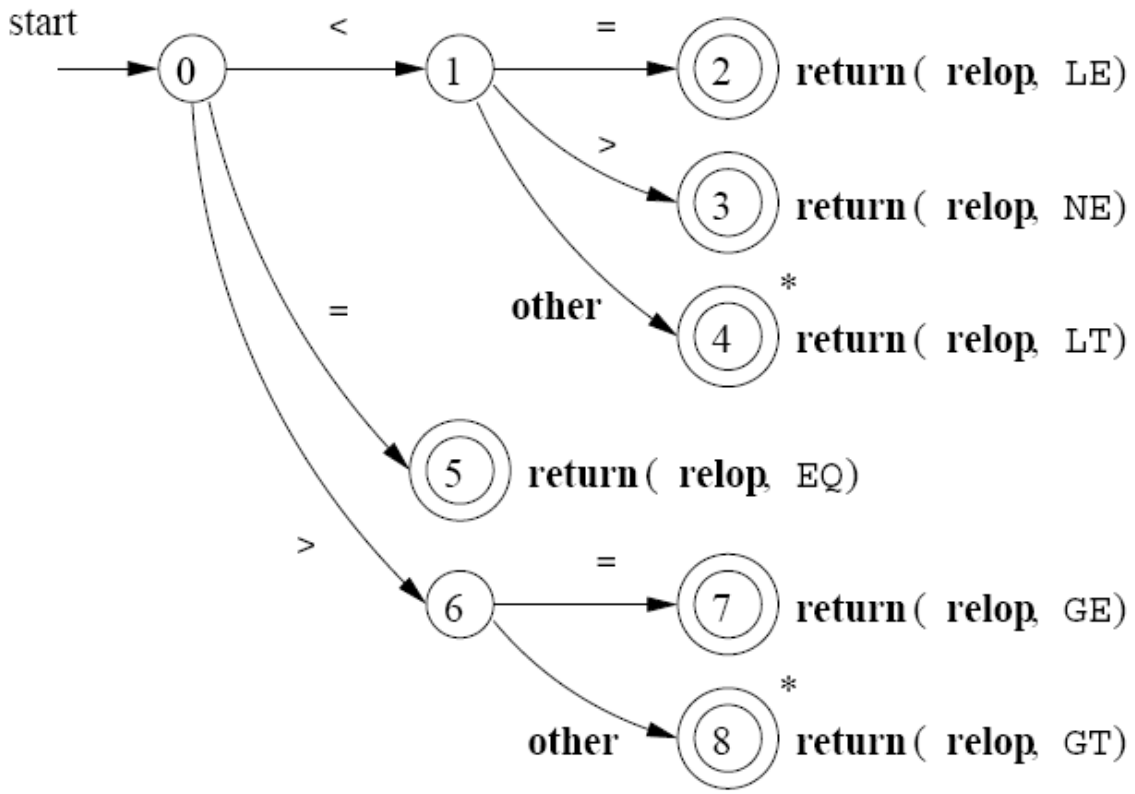


Figure 3.13: Transition diagram for **relop**

3.4.2 Recognition of Reserved Words and Identifiers

- *reserved* = **if** | **then** | **else** | **end** | **repeat** | **until** | **read** | **write**
- $id = letter\ (letter \mid digit)^*$

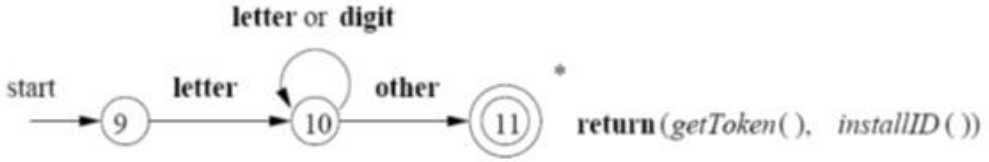


Figure 3.14: A transition diagram for **id**'s and keywords

- Solution1
  - Fig3.14, and then to look up the identifiers in a table of reserved words.
  - It implies using the principle of the longest substring to solve the ambiguity problem.
  - ambiguity : `ifthen` `if123` `<>`
  - **Principle of the longest substring**: When a string can be a single token or a sequence of several tokens, the single-token interpretation is preferred.
- Solution2: rule priority (Lex)

Recognition of Reserved Words

- Consider reserved words to be the same as identifiers, and then to look up the identifiers in a table of reserved words.

```
#define MAXRESERVED 8
/*lookup table of reserved words*/
static struct
{ char* str;
  TokenType tok;
} reservedWords[MAXRESERVED]
= {
      {"if",      IF},
      {"then",    THEN},
      {"else",    ELSE},
      {"end",     END},
      {"repeat",  REPEAT},
      {"until",   UNTIL},
      {"read",    READ},
      {"write",   WRITE}
  };
```

```
static TokenType reservedLookup(char* s)
{
  int i;
  for (i=0;i<MAXRESERVED;i++)
    if (!strcmp(s,reservedWords[i].str))
      return reservedWords[i].tok;
  return ID;
}
```

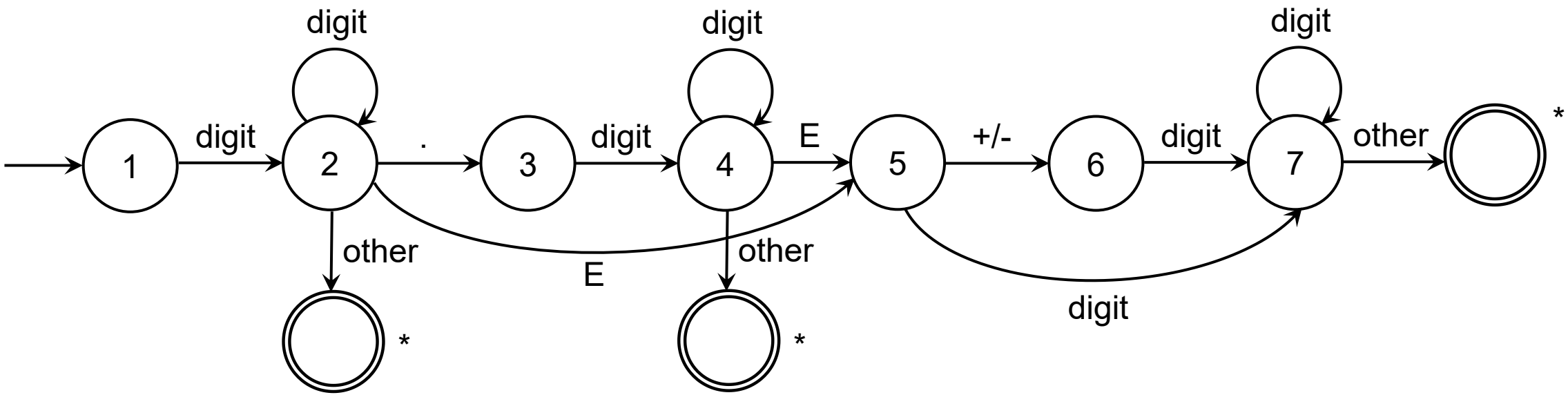
3.4.3 Recognition of Unsigned Numbers and Whitespace

- Unsigned numbers

$$digit = 0 \mid 1 \mid \ldots \mid 9$$

$$digits = digit\ digit^*$$

$$number = digits\ (.digits)\ ?\ (\mathrm{E}[+\text{-}]?\ digits)\ ?$$



Recognition of whitespace

- $whitespace = (\ newline \mid blank \mid tab)^{+}$
- $delim = newline \mid blank \mid tab$

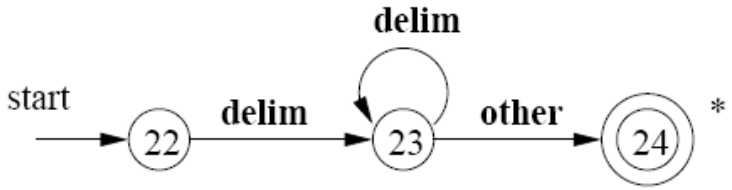
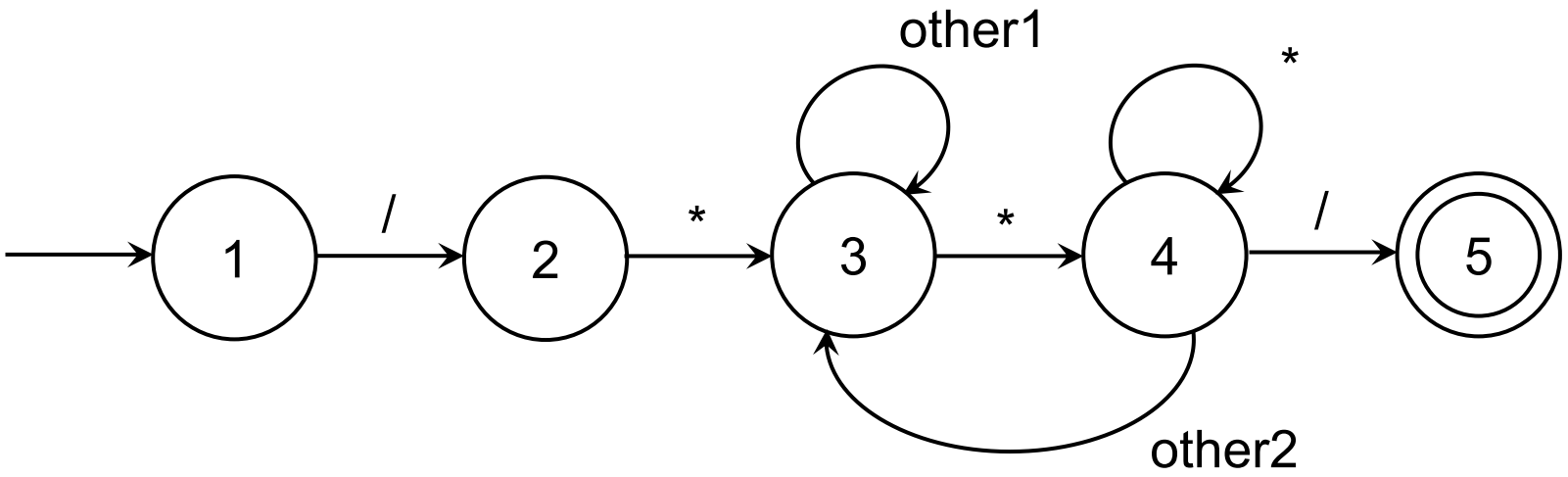


Figure 3.17: A transition diagram for whitespace

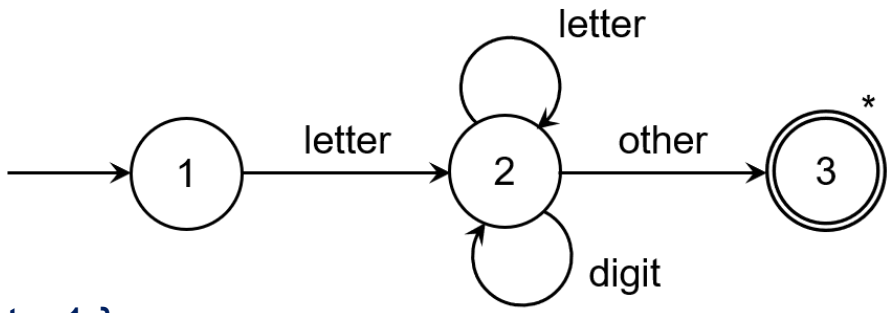
- Comments
  - Single line comments //
  - Multi-line comments /* */



3.4.4 Architecture of a Transition-Diagram-Based Lexical Analyzer

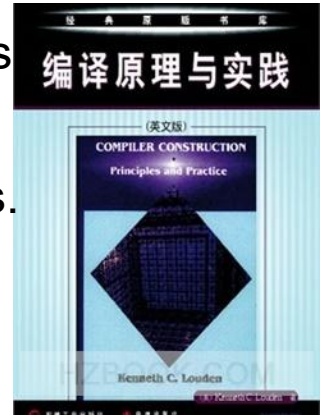
- There are several ways that a collection of transition diagrams can be used to build a lexical analyzer.

Implementation in Code - I

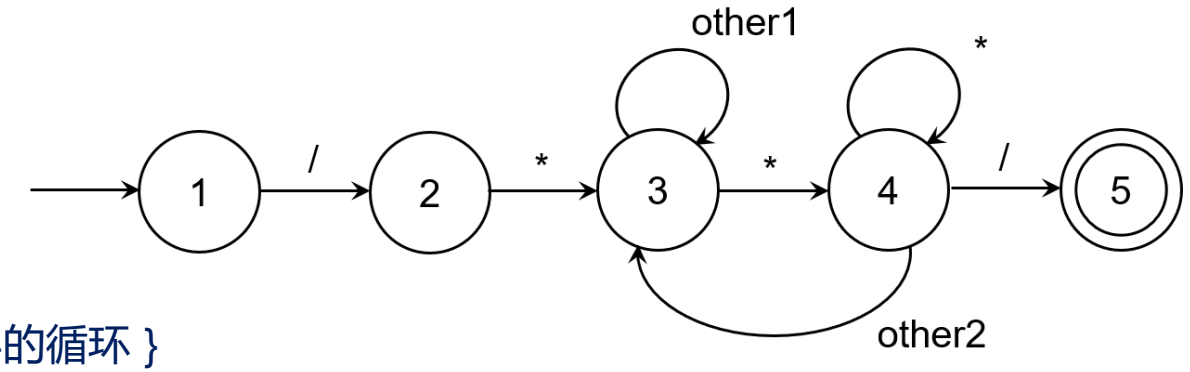


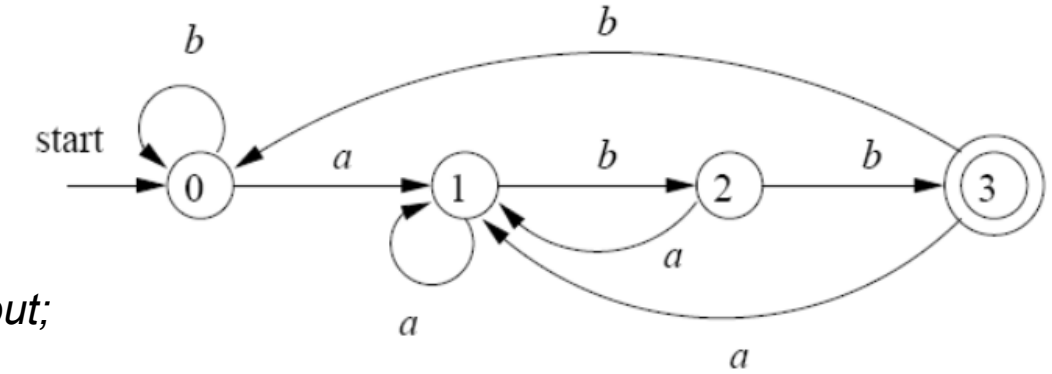

```
{ starting in state 1 }
if  the next character is a letter then
      advance the input;
      { now in state 2 }
      while the next character is a letter or a digit do
          advance the input; { stay in state 2 }
      end while;
      { go to state 3 without advancing the input}
      accept;
else
      { error or other cases }
end if;
```

- Drawbacks
  - It is ad hoc — that is, each DFA has to be treated slightly differently, and it is difficult to state an algorithm that will translate every DFA to code in this way.
  - The complexity of the code increases dramatically as the number of states rises, or, more specifically, as the number of different states along arbitrary paths rises.



```
{ state 1 }
if the next character is "/"  then
    advance the input; { state 2 }
    if the next character is " * " then
        advance the input; { state 3 }
        done := false;   { 利用done来处理涉及到状态3和4的循环 }
        while not done do
            while the next input character is not "*" do
                advance the input;
            end while;
            advance the input; { state 4 }
            while the next input character is "*" do advance the input;
            end while;
            if the next input character is "/" then done := true;  end if;
            advance the input;
        end while;
        accept; { state 5 }
    else { other processing }
    end if;
else { other processing }
end if;
```

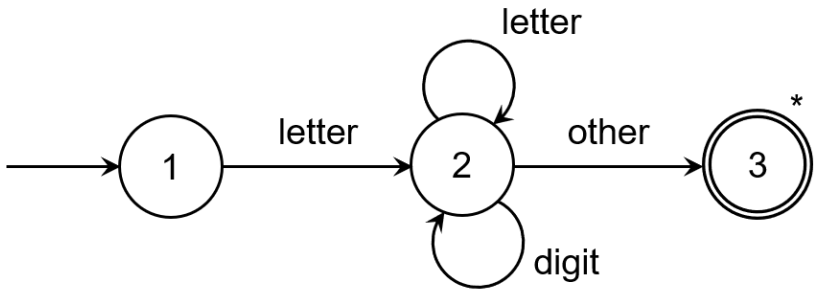


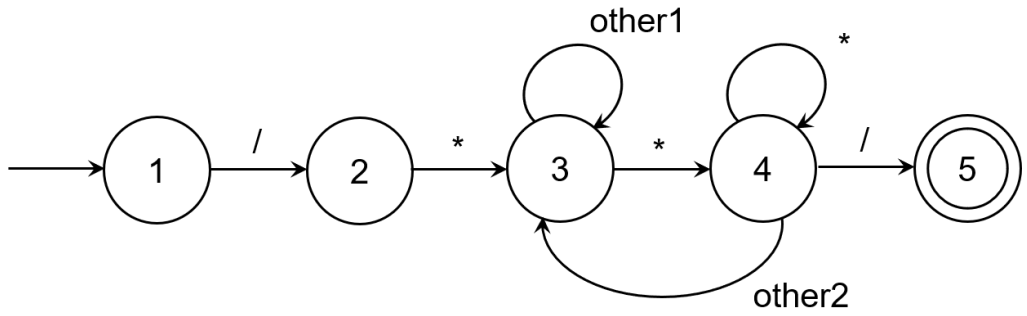


- Notice
  - the considerable increase in complexity, and
  - the need to deal with the loop involving state 3 an 4 by using the Boolean variable *done*.

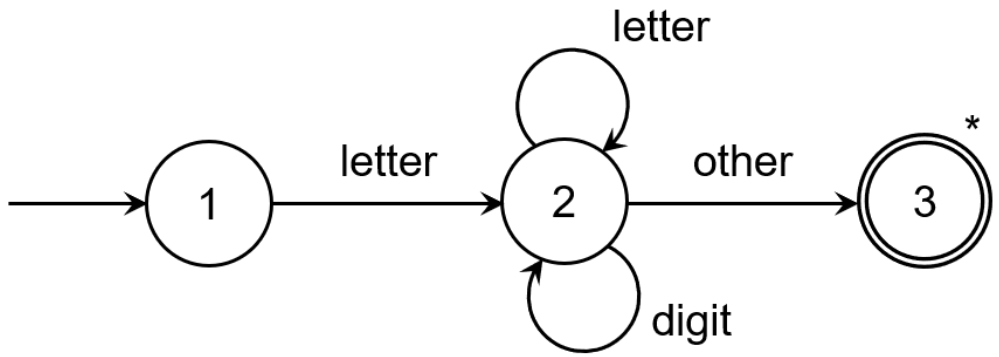
Implementation in Code - II

- A better method
  - current state : using a variable to maintain
  - transitions : a doubly nested case statement inside a loop, where
    - the 1st case : tests the current state
    - the 2nd case : tests the input character, given the state.

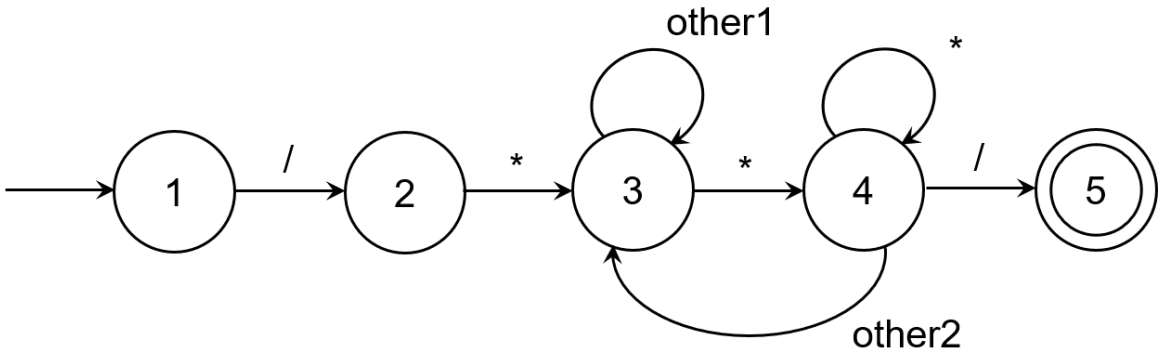




```
state := 1; { start }
while state = 1 or 2 do
    case state of
    1: case input character of
            letter: advance the input;
                    state := 2;
            else    state :=  ….{ error or other };
        end case;
    2: case input character of
            letter, digit: advance the input;
                    state := 2; { actually unnecessary }
            else    state := 3;
        end case;
    end case;
end while;
if state = 3 then accept else error;
```



```
state := 1;  { start }
while state = 1, 2, 3 or  4 do
  case state of
  1:  case input character of
        "/" :  advance the input ;
               state := 2;
        else state :=... { error or other };
        end case;
  2:  case input character of
        "*" :  advance the input ;
               state :=  3;
        else state :=.. .{ error or other };
        end case;
  3:  case input character of
        "*" :  advance the input ;
               state := 4;
        else  advance the input ;  {and stay in state 3};
        end  case;
```




```
  4:  case input character of
        "/" :  advance the input ;
               state := 5;
        "*" :  advance the input ;
               { and stay in state 4 }
        else  advance the input ;
               state := 3;
        end case;
  end case;
end while;
if state = 5 then accept else error ;
```

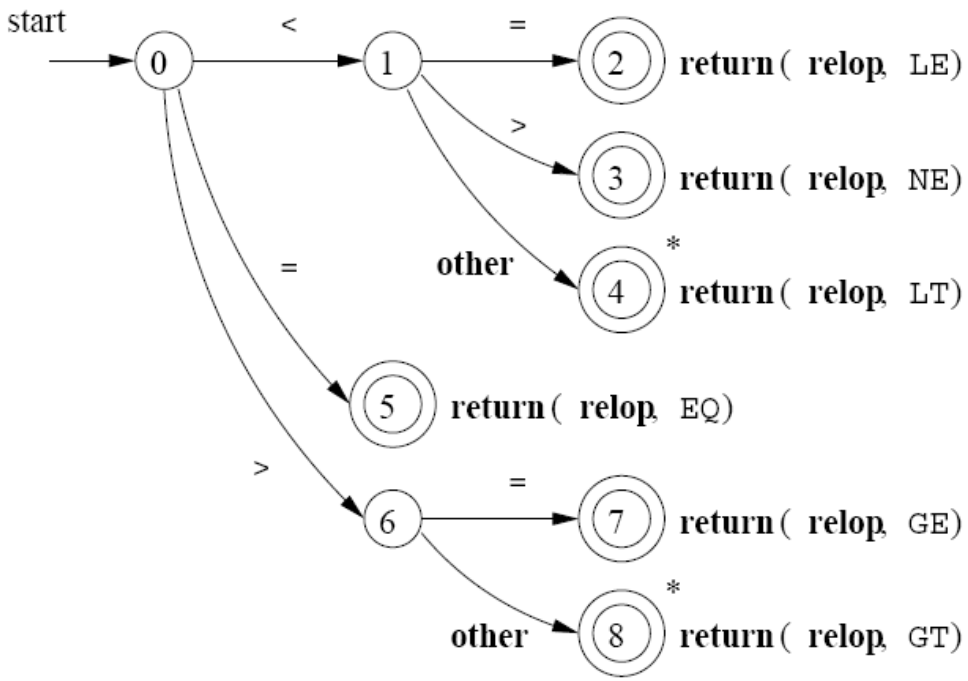


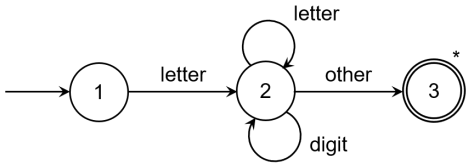
Figure 3.13: Transition diagram for **relop**

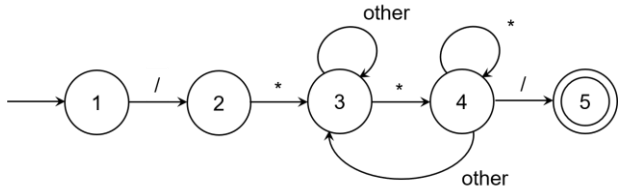
```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

Figure 3.18: Sketch of implementation of **relop** transition diagram

Implementation in Code - III

- Table-driven method ("generic" data structure and code)





- a transition array : $T[\,][\,]$
- a boolean array : $Advance[\,][\,]$
- a boolean array : $Accept[\,]$

input char / state	letter	digit	other	Accepting
1	2			no
2	2	2	[3]	no
3				yes

input char / state	/	*	other	Accepting
1	2			no
2		3		no
3	3	4	3	no
4	5	4	3	no
5				yes

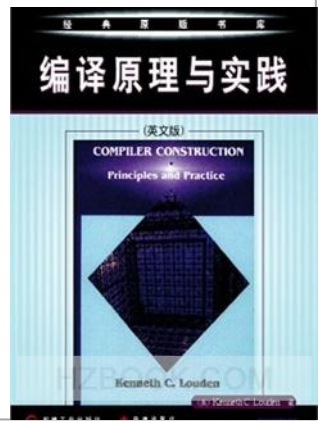
```
state := 1;
ch := next input character;
while not Accept[state] and not error(state) do
    newstate := T [state,ch];
    if Advance[state,ch] then  ch := next input char;
    state := newstate;
end while;
if Accept[state]  then accept;
```

Algorithm 3.18 vs. Implementation in Code - III

```
s = s0;
c = nextChar();
while ( c != eof ) {
        s = move(s, c);
        c = nextChar();
}
if ( s is in F ) return "yes";
else return "no";
```

Figure 3.27: Simulating a DFA

```
state := 1;
ch := next input character;
while not Accept[state] and not error(state) do
    newstate := T [state,ch];
    if Advance[state,ch]  then ch := next input char;
    state := newstate;
end while;
if Accept[state]  then accept;
```



Note

- Combine all the transition diagrams into one.

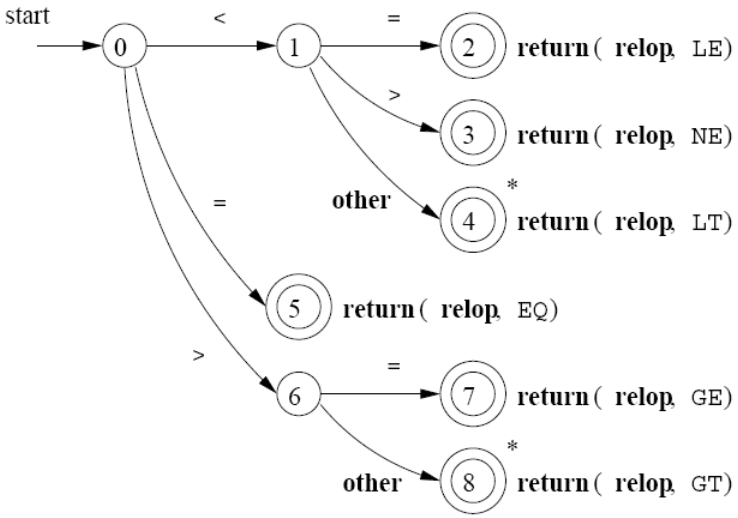


Figure 3.13: Transition diagram for **relop**

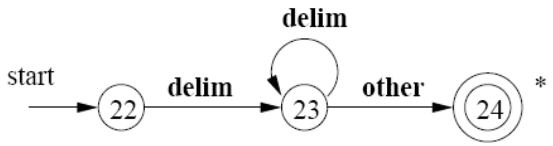


Figure 3.17: A transition diagram for whitespace

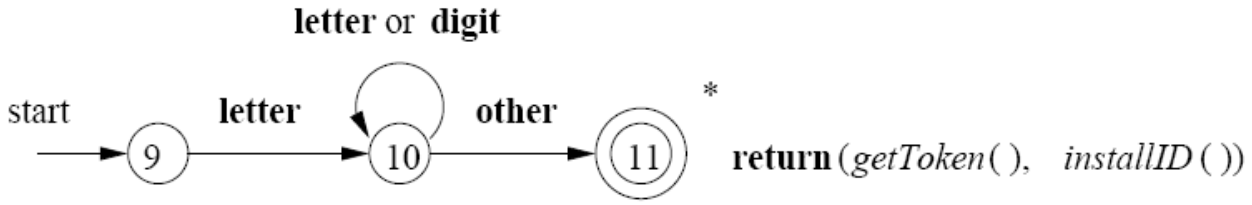


Figure 3.14: A transition diagram for **id**'s and keywords

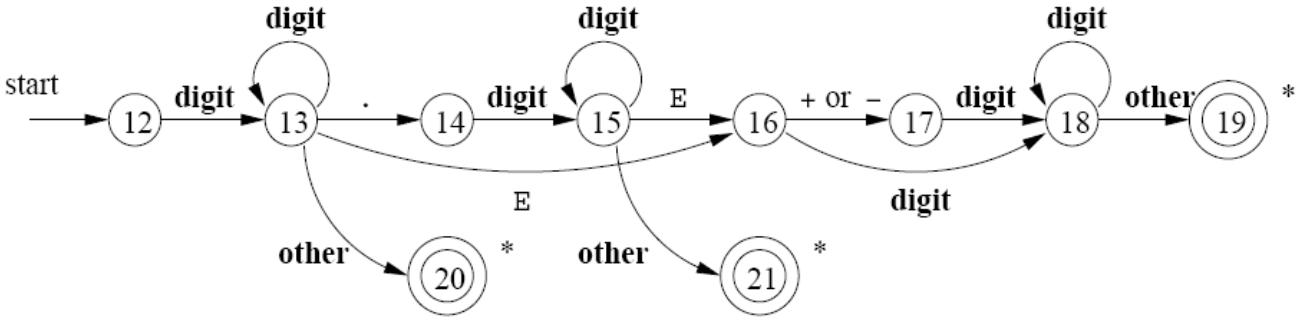
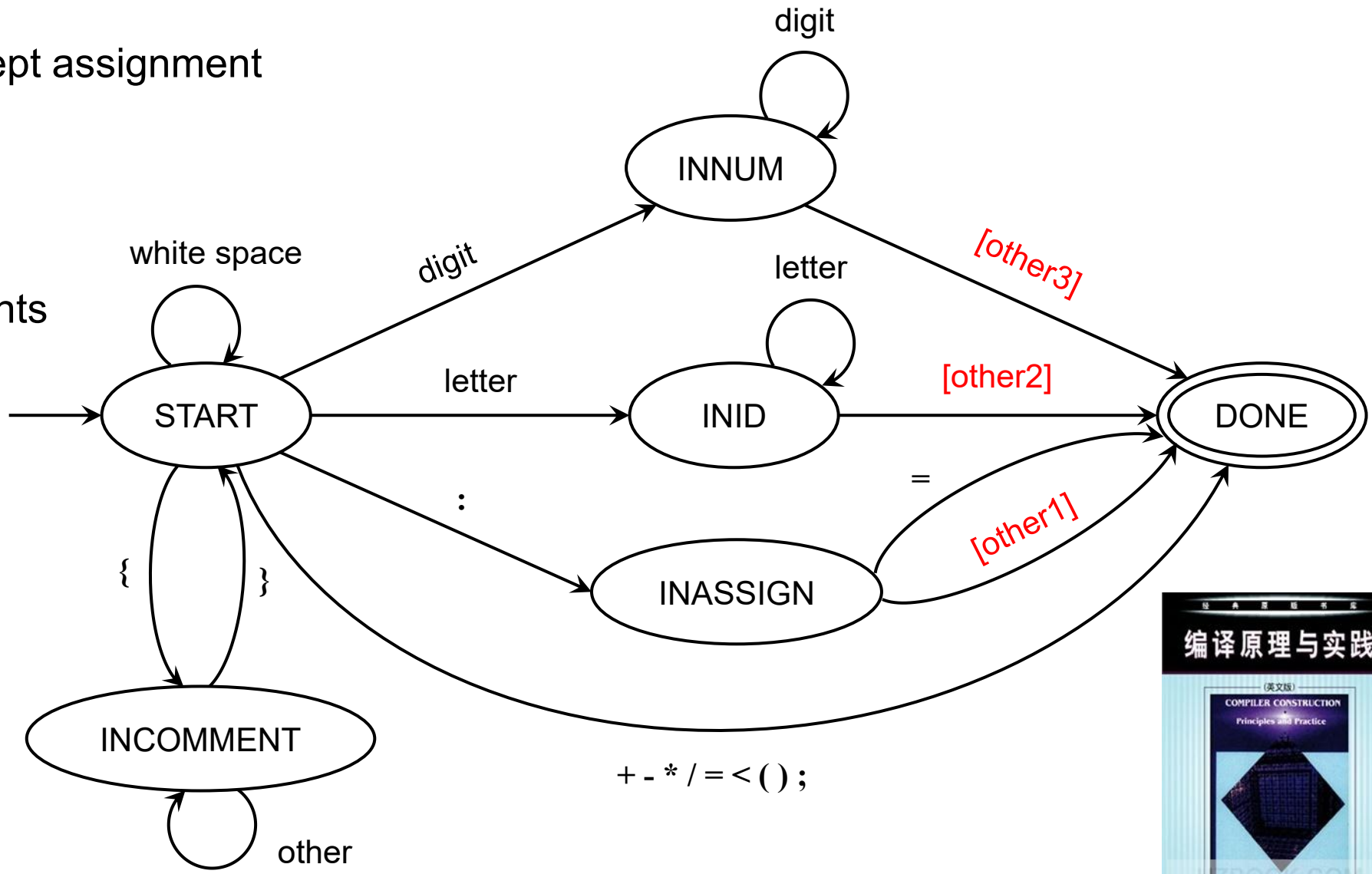


Figure 3.16: A transition diagram for unsigned numbers

Transition Diagram of a TINY Scanner

- special symbols except assignment
- assignment
- ID and number
- reserved words
- white space, comments
- error handling



Output of a TINY Scanner

```
{ Sample program
    in TINY language  -
    computes  factorial
}
read x; { input an integer }
if  0 < x then  { don't compute if x <= 0 }
    fact := 1;
    repeat
     fact := fact * x;
     x := x - 1
    until x = 0;
    write fact { output factorial of x }
end
```

```
// main.c
int EchoSource = TRUE;
int TraceScan = TRUE;
```

TINY COMPILATION: sample.tny

```
1: { Sample program
2:   in TINY language –
3:   computes factorial
4: }
5: read x; { input an integer }
5: reserved word: read
5: ID, name= x
5: ;
6: if 0<x then { don't compute if x<=0 }
6: reserved word: if
6: NUM, val= 0
6: <
6: ID, name= x
6: reserved word: then
7: fact := 1;
7: ID, name= fact
7: :=
7: NUM, val= 1
7: ;
8: repeat
8: reserved word: repeat
9: fact := fact * x;
9: ID, name= fact
9: :=
9: ID, name= fact
9: *
9: ID, name= x
9: ;
10:x := x - 1
10: ID, name= x
10: :=
10: ID, name=x
10: -
10: NUM, val = 1
11:until x = 0;
11: reserved word: until
11: ID, name= x
11: =
11: NUM, val= 0
11: ;
12: write fact { output factorial of x }
12: reserved words: write
12: ID, name= fact
13:end
13: reserved word: end
14: EOF
```

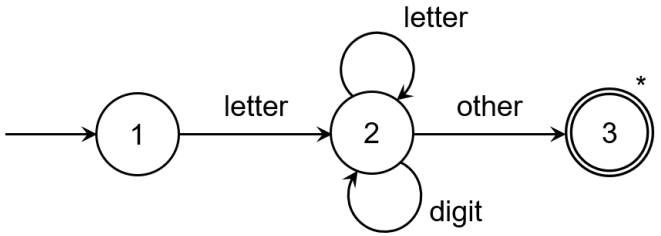
3.6 Finite Automata

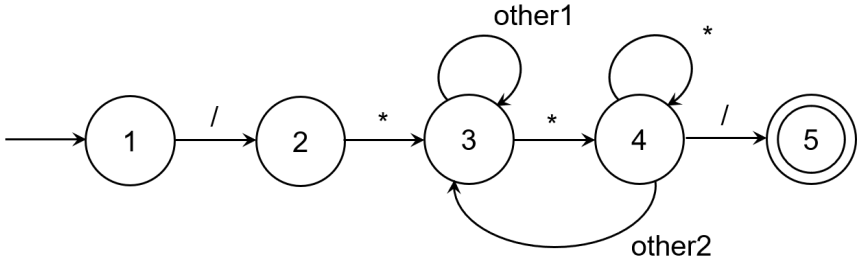
- $S = \{ 1, 2, 3 \}, C = \{ a, b \}$

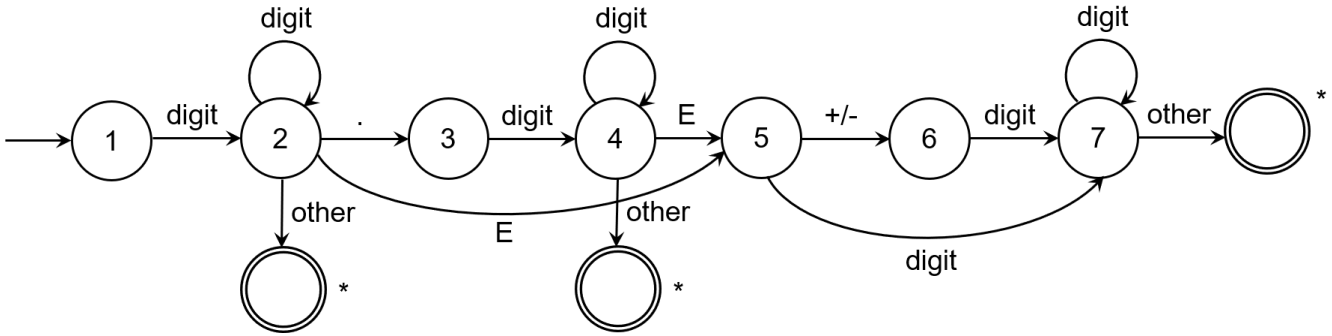
  (1) $S \times C =$

  (2) $2^S =$

Common Characteristics







1. S
2. Σ
3. f
4. s_0
5. F

$<S, \Sigma, f, s_0, F>$

3.6.4 Deterministic Finite Automata

- A DFA consists of $<S, \Sigma, f, s_0, F>$
  1. a finite set of states S
  2. an alphabet Σ
  3. a transition function $f: S \times \Sigma \rightarrow S$
  4. a start state s_0 from S
  5. a set of accepting states F from S.

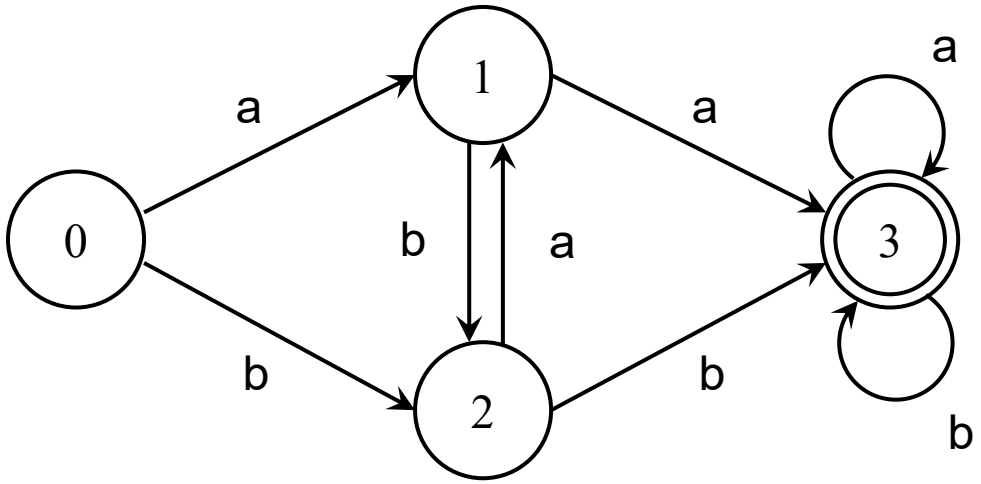
一个DFA包括：
一个有限的状态集合S
一个符号域
一个转移方程
一个包括在S中的开始状态
一系列包括在S中的接受状态F

Example

- DFA $M = <\{0, 1, 2, 3\}, \{a, b\}, f, 0, \{3\}>$，其中，$f$ 定义如下：

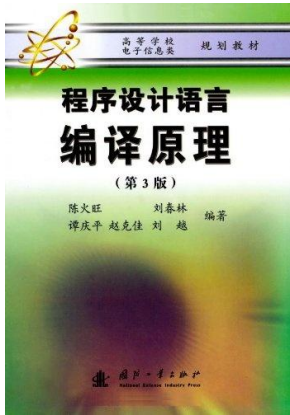
$$f(0, a) = 1 \quad f(0, b) = 2$$
$$f(1, a) = 3 \quad f(1, b) = 2$$
$$f(2, a) = 1 \quad f(2, b) = 3$$
$$f(3, a) = 3 \quad f(3, b) = 3$$

- 状态转移图



- 状态转移矩阵 / 状态转移表

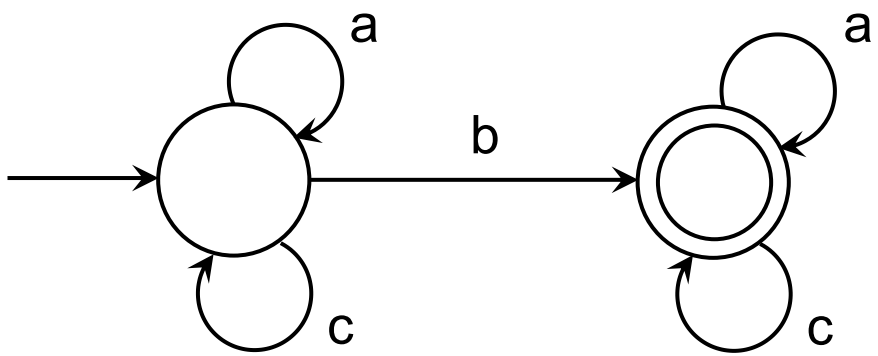
State	a	b
0	1	2
1	3	2
2	1	3
3	3	3



Example

- $\Sigma = \{ a, b, c \}$

1. the set of strings that contain exactly one b

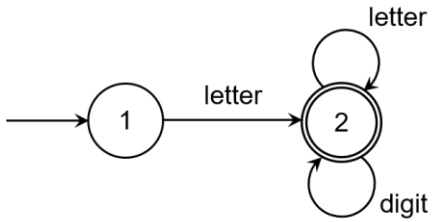


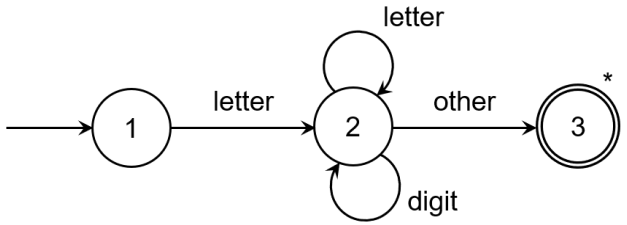
2. the set of strings that contain at most one b

- The DFA is equivalent to $(a|c)^*b(a|c)^*$.
- 正则表达式和有穷自动机之间存在等价关系。

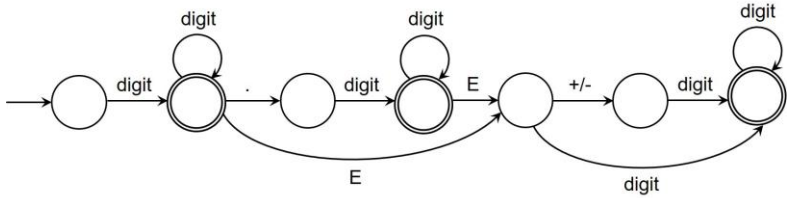
DFA for A Scanner

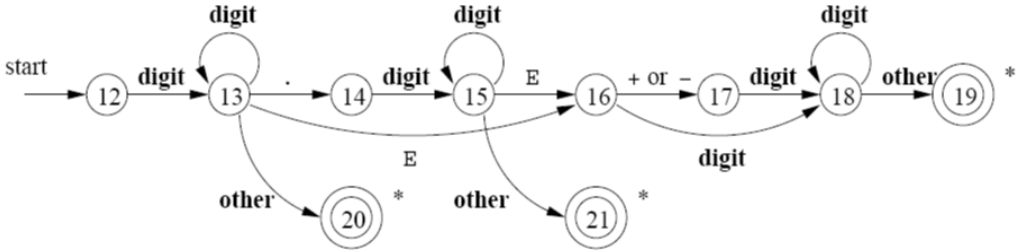
- Note: lookahead
  - *letter* (*letter* | *digit*)*



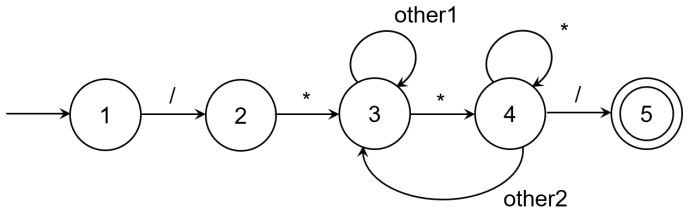


  - *number* = *digits* (.*digits*) ? (E[+-]? *digits*) ?





  - /* … */



3.6.1 Nondeterministic Finite Automata

- An NFA consists of $<S, \Sigma, f, s_0, F>$
  1. a finite set of states S
  2. an alphabet Σ
  3. a transition function f: $S \times (\Sigma \cup \{\varepsilon\}) \rightarrow \rho(S)$ or $S \times \Sigma^* \rightarrow 2^S$
  4. a start state s_0 from S
  5. a set of accepting states F from S.

3.6.2 Transition Tables

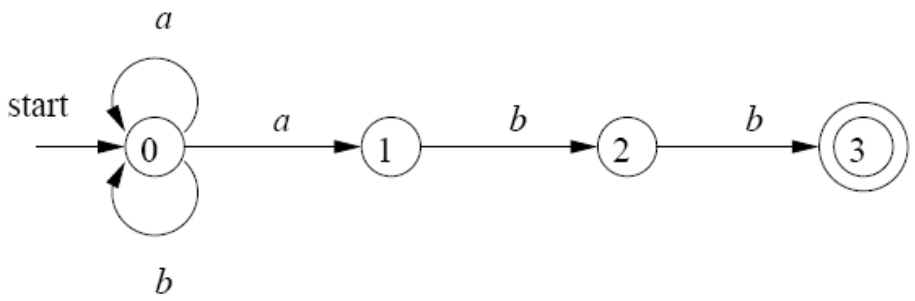


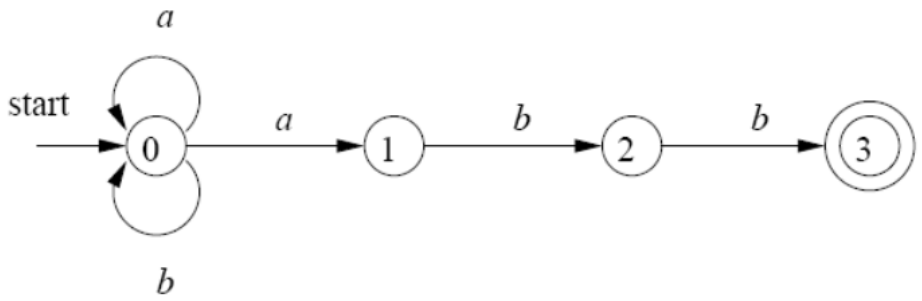
Figure 3.24: A nondeterministic finite automaton

STATE	a	b	ϵ
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

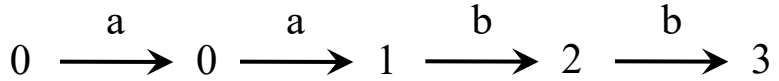
Figure 3.25: Transition table for the NFA of Fig. 3.24

3.6.3 Acceptance of Input Strings by Automata

- 一个FA接受输入字符串 x，当且仅当对应的转换图中存在一条从开始状态到某个接受状态的路径，使得该路径中各条边上的标号组成字符串 x。



- 该NFA能够接受（识别）串 aabb。



- 一个FA能够接受的所有字符串的集合称为该FA定义（或接受）的语言。

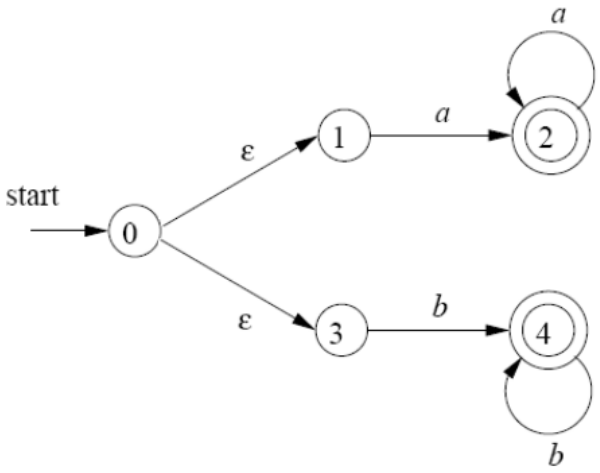
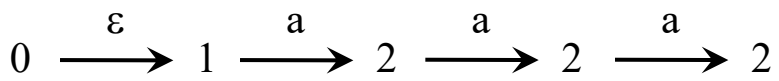


Figure 3.26: NFA accepting $\mathbf{aa}^*|\mathbf{bb}^*$

- 该NFA能够接受（识别）串 aaa。



- 路径中的 ε 被忽略，因为空串不会影响到根据路径构建得到的符号串。

Exercises

- Exercise 3.6.3: For the NFA of Fig 3.29, indicate all the paths labeled *aabb*. Does the NFA accept *aabb*?

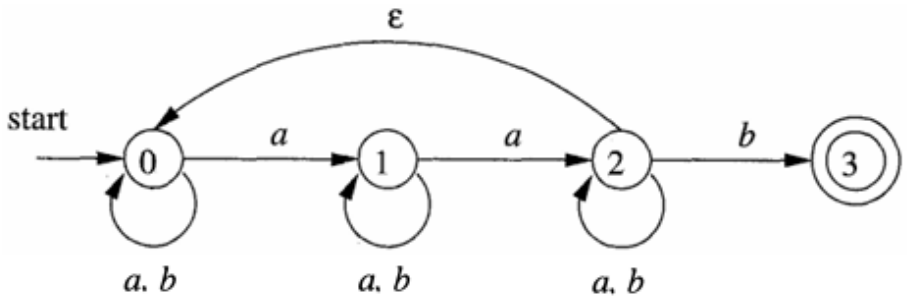


Figure 3.29: NFA for Exercise 3.6.3

能够接受aabb

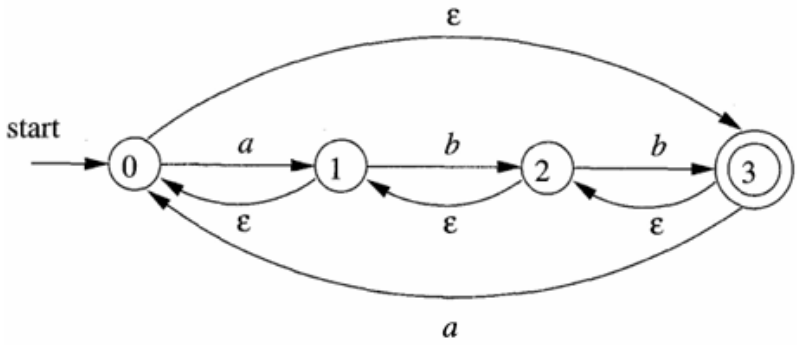


Figure 3.30: NFA for Exercise 3.6.4

能够接受aabb

- Exercise 3.6.4 : Repeat Exercise 3.6.3 for the NFA of Fig 3.30.

Difference between DFA and NFA

- A DFA is a special case of an NFA where:
  1. there are no move on input ε in a DFA, and
  2. DFA: $S \times \Sigma \rightarrow S$

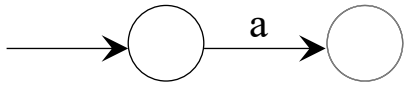
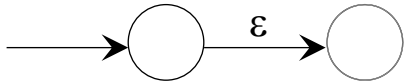
     NFA: $S \times (\Sigma \cup \{\varepsilon\}) \rightarrow \rho(S)$ or $S \times \Sigma^* \rightarrow 2^S$

- Finite Automata vs. Transition Diagrams
  - Finite automata are essentially graphs, like transition diagrams, with a few differences:

    1. recognizer; 2. NFA and DFA.

From RE to NFA

- Design regular expressions and the equivalent NFAs for the following language:
  - $\Sigma = \{ a, b \}$, all strings that contain consecutive two a's or consecutive two b's.

Basis	a	
	ε	
Induction	a | b	
	ab	
	a*	
	(a|b)*	
	(ab)*	
	a*b*	
	(a|b)*(ab)*	

Exercises

- Design finite automata (deterministic or nondeterministic) for each of the following languages.
  - All strings of decimal integers that are divisible by 5 (can start with 0).

! Exercise 3.3.5: Write regular definitions for the following languages:

a) All strings of lowercase letters that contain the five vowels in order.

b) All strings of lowercase letters in which the letters are in ascending lexicographic order.

(each letter appears at least once / including the empty string)

Exercise 3.6.2: Design finite automata (deterministic or nondeterministic) for each of the languages of Exercise 3.3.5.